Smart FInance Presentation

Banking Data Analysis

MySQL Capstone Project



Presented By: **Hazra Shaikh**

www.reallygreatsite.com

Project Overview

A complete final SQL project in which you build and analyze a simulated banking database. It is designed to exercise the full syllabus: DDL, DML, DQL, constraints, joins, aggregation, functions, CASE, subqueries, window functions, CTEs, views, data cleaning, indexes, EXPLAIN and transactions.

Business Scenario

You are a junior SQL/Data Analyst at a bank. The bank needs a relational system for customers, branches, accounts, beneficiaries, transactions, loans, loan payments, cards and employees. Management wants analytical answers about customer activity, branch performance, transaction trends, loan risk and high-value customers.





Objectives

- Design a relational banking database.
- Create tables with suitable data types and constraints.
- Import and validate the supplied CSV data.
- Write basic and advanced SQL queries.
- Use joins, aggregation, subqueries, CTEs and window functions.
- Clean and validate data using SQL.
- Create reusable views.
- Use indexes and EXPLAIN for performance analysis.
- Demonstrate COMMIT, ROLLBACK, SAVEPOINT and ACID concepts.
- Present business insights supported by SQL.

Technology & Dataset

Item	Requirement
Database	MySQL 8.0+ / MariaDB 10.11+
Tool	MySQL Workbench recommended
Database name	banking_system
Input	9 CSV files
Records	6,400+ synthetic records
Output	SQL scripts + ER diagram + final report

Dataset Files

File	Rows	Purpose
customers.csv	500	Customer master (includes a few intentional duplicate/missing values for the cleaning module)
branches.csv	30	Branch master
accounts.csv	586	Customer accounts
beneficiaries.csv	401	Saved beneficiaries for transfers
transactions.csv	3,000	Transaction history (includes a few negative/suspicious amounts)
loans.csv	360	Loans
loan_payments.csv	941	Loan repayment history
cards.csv	443	Debit/credit cards
employees.csv	150	Employees and manager hierarchy (self-referencing)

Table Responsibilities

Table	Purpose	Keys
customers	Customer master	customer_id PK
branches	Branch master	branch_id PK
accounts	Bank accounts	account_id PK; customer_id, branch_id FK
beneficiaries	Saved beneficiaries	beneficiary_id PK; customer_id FK
transactions	Transaction history	transaction_id PK; account_id FK
loans	Loan master	loan_id PK; customer_id, branch_id FK
loan_payments	Loan repayment history	payment_id PK; loan_id FK
cards	Card information	card_id PK; customer_id, account_id FK
employees	Employee hierarchy	employee_id PK; branch_id FK; manager_id self-reference

Project Questions

1. List the number of customers living in each city, ordered from most to fewest.

```
SELECT city,
COUNT(customer_id) AS
No_of_Customer
FROM customers
GROUP BY city
ORDER BY No_of_Customer
DESC;
```

city	No_of_Customer
Patna	33
Nagpur	33
Hyderabad	31
Delhi	31
Indore	30
Lucknow	29
Kolkata	29

Project Questions

2. Find the total balance held in accounts, grouped by account_type.

```
SELECT account_type, SUM(balance) AS
total_balance
FROM accounts
GROUP BY account_type;
```

account_type	total_balance
Salary	35797490.55
Current	35326473.27
Savings	38387721.91
Fixed Deposit	83383533.44

Project Questions

3. Find branches that manage more than 15 accounts.

```
SELECT b.branch_name,
COUNT(b.branch_id) AS
Accounts_count
FROM branches b
JOIN accounts a
ON a.branch_id = b.branch_id
GROUP BY b.branch_id,
b.branch_name
HAVING Accounts_count > 15;
```

branch_name	Accounts_count
Surat Branch 1	18
Mysuru Branch 2	24
Lucknow Branch 3	21
Mumbai Branch 4	21
Nagpur Branch 5	26
Chandigarh Branch 6	18
Bengaluru Branch 7	17

Project Questions

4. For every branch, show how many accounts it holds and the branch's total balance (include branches with zero accounts).

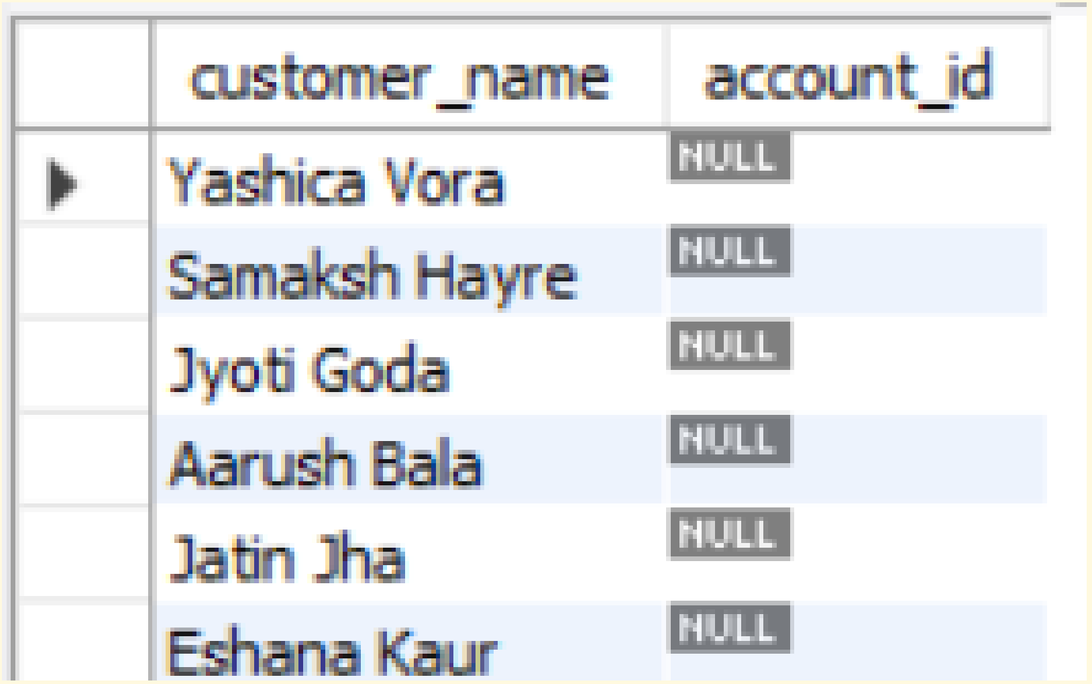
```
SELECT b.branch_name,
COUNT(a.account_id) AS
Accounts_count,
SUM(a.balance) AS
branch_total_bal
FROM branches b
LEFT JOIN accounts a
ON a.branch_id = b.branch_id
GROUP BY b.branch_name;
```

branch_name	Accounts_count	branch_total_bal
Surat Branch 1	18	4171376.83
Mysuru Branch 2	24	8069942.33
Lucknow Branch 3	21	8002557.33
Mumbai Branch 4	21	6646786.98
Nagpur Branch 5	26	996621.55
Chandigarh Branch 6	18	5883727.45
Bengaluru Branch 7	17	5687583.35

Project Questions

5. Find customers who do not currently hold any account.

```
SELECT CONCAT(c.first_name,'
',c.last_name) AS
customer_name,
a.account_id
FROM customers c
LEFT JOIN accounts a
ON c.customer_id =
a.customer_id
WHERE account_id IS NULL;
```



	customer_name	account_id
▶	Yashica Vora	NULL
	Samaksh Hayre	NULL
	Jyoti Goda	NULL
	Aarush Bala	NULL
	Jatin Jha	NULL
	Eshana Kaur	NULL

Project Questions

6. For every employee, show their name, designation, and their manager's name (if they have one).

```
SELECT e.employee_name,
e.designation,
m.employee_name AS
manager_name
FROM employees e
LEFT JOIN employees m
ON e.manager_id =
m.employee_id;
```

employee_name	designation	manager_name
Kashish Chatterjee	Regional Head	NULL
Darpan Jha	Regional Head	NULL
Dipta Shroff	Regional Head	NULL
Janaki Mistry	Regional Head	NULL
Yagnesh Bumb	Regional Head	NULL
Hemal Sen	Branch Manager	Dipta Shroff
Jacob Vala	Branch Manager	Darpan Jha
Max Suresh	Branch Manager	Kashish Chatterjee
Dipta Oak	Branch Manager	Dipta Shroff

Project Questions

7. Categorize every account into a balance_band of 'Low' (under ₹10,000), 'Medium' (₹10,000–100,000) or 'High' (over ₹100,000).

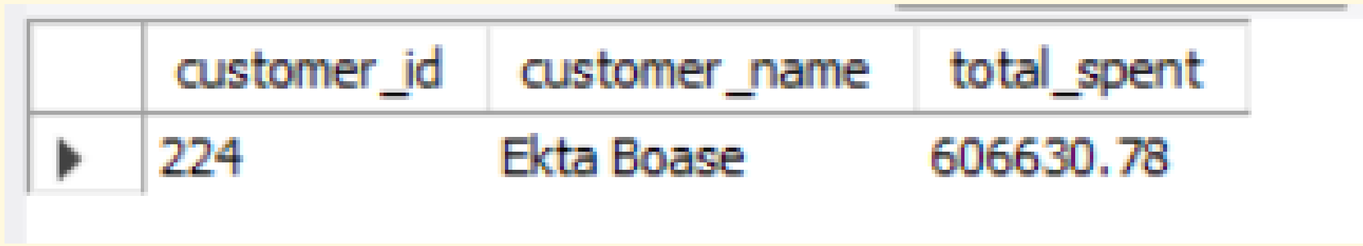
```
SELECT account_type, balance,
CASE
 WHEN balance < 10000 THEN "Low"
 WHEN balance BETWEEN 10000 AND
100000 THEN "Medium"
   ELSE "High"
END AS balance_band
FROM accounts ;
```

	account_type	balance	balance_band
▶	Salary	14750.49	Medium
	Salary	47440.88	Medium
	Current	359232.97	High
	Savings	183396.09	High
	Current	171306.69	High
	Savings	146794.97	High
	Salary	136213.13	High
	Current	396079.59	High

Project Questions

8. Find the single customer who has spent the most in total transaction amount.

```
SELECT c.customer_id, CONCAT(c.first_name,' ',c.last_name) AS customer_name,
SUM(t.amount) AS total_spent
FROM customers c
JOIN accounts a
ON c.customer_id = a.customer_id
JOIN transactions t
ON t.account_id = a.account_id
WHERE t.transaction_type NOT IN ('Deposit', 'Transfer')
GROUP BY c.customer_id
ORDER BY total_spent DESC
LIMIT 1;
```



customer_id	customer_name	total_spent
224	Ekta Boase	606630.78

Project Questions

9. Find employees who earn more than the average salary of their own branch.

```
SELECT e.employee_name,
e.salary, e.branch_id
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.branch_id =
e.branch_id
);
```

employee_name	salary	branch_id
Hemal Sen	98734.00	1
Jacob Vala	102717.00	2
Max Suresh	103707.00	3
Dipta Oak	103358.00	4
Tanmayi Biswas	88376.00	5
Urvi Bhattacharyya	103717.00	6
Sanaya Mane	97202.00	7
Arin Shenoy	102948.00	8

Project Questions

10.Find the top 3 customers by account balance in every city.

```
WITH ranked AS (
    SELECT
        c.customer_id,
        CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
        c.city,
        SUM(a.balance) AS total_balance,
        ROW_NUMBER() OVER (
            PARTITION BY c.city
            ORDER BY SUM(a.balance) DESC
        ) AS rnk
    FROM customers c
    JOIN accounts a
        ON c.customer_id = a.customer_id
    GROUP BY c.customer_id, c.city
)
SELECT *
FROM ranked
WHERE rnk <= 3;
```

customer_id	customer_name	city	total_balance	rnk
100	Netra Gour	Ahmedabad	1010094.84	1
385	Rehaan Shere	Ahmedabad	883681.46	2
392	Avni Dayal	Ahmedabad	875028.87	3
102	Pushti Sethi	Bengaluru	518268.70	1
299	Sneha Chaudhuri	Bengaluru	486048.55	2
82	Barkha Kakar	Bengaluru	464316.66	3
311	Irya Khosla	Bhopal	1263414.07	1
432	Chakrika Goda	Bhopal	736703.53	2
469	Alka Panchal	Bhopal	632046.28	3

Project Questions

11.For each month, show the total transaction amount and compare it with the previous month's total.

```
SELECT
  transaction_month,
  total_amount,
  LAG(total_amount) OVER (ORDER BY
transaction_month) AS
previous_month_amount
FROM (
  SELECT
    DATE_FORMAT(transaction_date, '%Y-
%m') AS transaction_month,
    SUM(amount) AS total_amount
  FROM transactions
  GROUP BY DATE_FORMAT(transaction_date,
'%Y-%m')
) AS monthly_data;z
```

transaction_month	total_amount	previous_month_amount
2019-02	93367.89	NULL
2019-04	22675.42	93367.89
2019-06	44817.44	22675.42
2019-11	22789.37	44817.44
2019-12	65504.92	22789.37
2020-01	81736.02	65504.92
2020-02	84440.91	81736.02
2020-06	74520.49	84440.91

Project Questions

12.Create a view called customer_account_summary showing each customer's total number of accounts and total balance.

```
CREATE VIEW
customer_account_summary AS
SELECT CONCAT(c.first_name, ' ',
c.last_name) AS customer_name,
COUNT(a.account_id) AS
total_no_of_acc,
COALESCE(SUM(a.balance),0) AS
total_bal
FROM customers c
LEFT JOIN accounts a
ON c.customer_id = a.customer_id
GROUP BY c.customer_id, c.first_name,
c.last_name;

SELECT * FROM
customer_account_summary;
```

customer_name	total_no_of_acc	total_bal
Kalpit Ravi	1	14750.49
Ekaraj Hans	1	47440.88
Lopa Nigam	1	359232.97
Devansh Bali	1	183396.09
Yutika Chopra	2	318101.66
Dayita Chaudhary	1	136213.13
Zansi Keer	1	396079.59
Anay Nigam	1	486388.48

Project Questions

13.Using a recursive CTE, produce the full employee reporting hierarchy starting from each Regional Head.

```
WITH RECURSIVE employee_hierarchy AS (
SELECT
employee_id, employee_name, designation,
manager_id, 1 AS level
FROM employees
WHERE designation = 'Regional Head'

UNION ALL

SELECT e.employee_id, e.employee_name,
e.designation, e.manager_id,
eh.level + 1
FROM employees e
JOIN employee_hierarchy eh
ON e.manager_id = eh.employee_id
)
SELECT *
FROM employee_hierarchy
ORDER BY level, employee_id;
```

employee_id	employee_name	designation	manager_id	level
1	Kashish Chatterjee	Regional Head	NULL	1
2	Darpan Jha	Regional Head	NULL	1
3	Dipta Shroff	Regional Head	NULL	1
4	Janaki Mistry	Regional Head	NULL	1
5	Yagnesh Bumb	Regional Head	NULL	1
6	Hemal Sen	Branch Manager	3	2
7	Jacob Vala	Branch Manager	2	2
8	Max Suresh	Branch Manager	1	2

Project Questions

14.identify duplicate customer records based on a normalized (trimmed, uppercased) email address.

```
SELECT
   UPPER(TRIM(email)) AS
normalized_email,
   COUNT(*) AS
duplicate_count
FROM customers
WHERE email IS NOT NULL
GROUP BY
UPPER(TRIM(email))
HAVING COUNT(*) > 1;
```

normalized_email	duplicate_count
PRANIT.SETHI31@EXAMPLE.COM	2
GAVIN.SEN77@EXAMPLE.COM	3
EKBAL.KHANNA107@EXAMPLE.COM	2
MUGDHA.CONTRACTOR116@EXAMPLE.COM	2
OMAJA.SIDHU182@EXAMPLE.COM	2
TRIPTI.BOSE199@EXAMPLE.COM	2
MAANAV.SHANKAR217@EXAMPLE.COM	2

Project Questions

15.Identify any transactions with a negative amount.

```
SELECT transaction_id,
transaction_type, amount
FROM transactions
WHERE amount < 0;
```

transaction_id	transaction_type	amount
500001	Withdrawal	-9404.87
500098	Deposit	-67718.92
500195	POS Purchase	-48610.96
500292	Transfer	-40181.79
500389	Bill Payment	-68523.16
500486	ATM Withdrawal	-40445.95
500583	ATM Withdrawal	-66596.45
500680	Withdrawal	-52860.36
500777	Transfer	-57601.68
500874	Withdrawal	-31613.21

Project Questions

16.Use EXPLAIN to see how MySQL executes a lookup of transactions for a specific account_id, then add an index and compare the plan.

```
EXPLAIN
SELECT * FROM transactions
WHERE account_id = 101;

CREATE INDEX idx_account_id
ON transactions(account_id);
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	transactions	NULL	ref	idx_account_id	idx_account_id	5	const	1	100.00	NULL

Project Questions

17.Branches with the most overdue/defaulted loans?

```
SELECT b.branch_id, b.branch_name,
COUNT(l.loan_id) AS
overdue_defaulted_loans
FROM branches b
JOIN loans l
ON b.branch_id = l.branch_id
WHERE l.status IN
('Overdue','Defaulted')
GROUP BY b.branch_id,
b.branch_name
ORDER BY overdue_defaulted_loans
DESC;
```

branch_id	branch_name	overdue_defaulted_loans
29	Kochi Branch 29	6
23	Kochi Branch 23	5
1	Surat Branch 1	4
11	Patna Branch 11	4
17	Nagpur Branch 17	4
25	Coimbatore Branch 25	4
14	Nagpur Branch 14	3

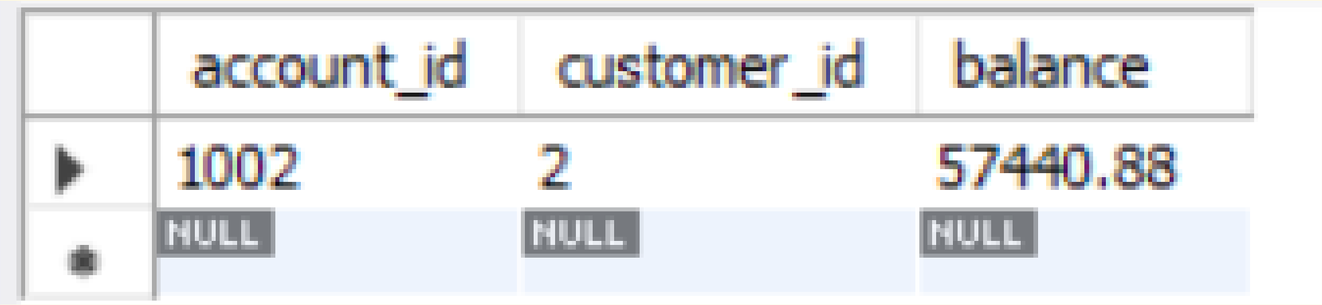
Project Questions

18.Write a transaction that transfers ₹5,000 from one account to another, ensuring both the debit and the credit succeed together or not at all.

```
START TRANSACTION;
-- Credit ₹5,000 to account 1002
UPDATE accounts
SET balance = balance + 5000
WHERE account_id = 1002;
-- Debit ₹5,000 to account 1002
UPDATE accounts
SET balance = balance - 5000
WHERE account_id = 1002;

COMMIT;

SELECT account_id, customer_id, balance
FROM accounts
WHERE account_id IN (1001, 1002);
```



account_id	customer_id	balance
1002	2	57440.88
NULL	NULL	NULL

Project Questions

19.Add a SAVEPOINT before the transfer and show a ROLLBACK TO SAVEPOINT that undoes only the transfer, leaving any earlier work in the same session untouched.

```
START TRANSACTION;
-- Make an earlier change
UPDATE accounts
SET balance = balance - 4000
WHERE account_id = 1003;

-- Create savepoint
SAVEPOINT before_transfer;
```

account_id	customer_id	balance
1003	3	360232.97
NULL	NULL	NULL

```
UPDATE accounts
SET balance = balance + 5000
WHERE account_id = 1003;

-- Undo only the transfer
ROLLBACK TO SAVEPOINT before_transfer;

-- Save the earlier change
COMMIT;
```

account_id	customer_id	balance
1003	3	365232.97
NULL	NULL	NULL

Project Questions

20.Explain, in your own words, how your transaction satisfies each of the four ACID properties: Atomicity, Consistency, Isolation and Durability.

ACID Properties

1. Atomicity: A transaction is completed fully or not at all. If one operation fails, all changes are rolled back.

2. Consistency: A transaction maintains the accuracy and integrity of the database. It ensures all database rules and constraints are followed.

3. Isolation: Multiple transactions execute independently without interfering with each other. Incomplete changes are not visible to other transactions.

4.Durability: Once a transaction is committed, its changes are permanently saved. They remain saved even if the system crashes.

Banking Data Analysis

Thank you so much

https://github.com/Hazra-Shaikh/